

Project 3

Predictive Modeling in Marketing Analytics

Predicting Ad-Campaign Conversion with Regularized Logistic Regression (Lasso and Ridge)

Student: _____

Course: _____

Instructor: _____

Date: October 1, 2026

Deliverables: Project3_Regularized_Conversion_Model.ipynb (full, executed analysis) and this report.

1. Introduction

A digital marketing team ran a campaign as an A/B test. Most users were shown product advertisements (the “ad” group), while a small control group was shown a public-service announcement (“psa”) in the same placements. Only 2.52% of users converted, meaning they bought the product. The team needs to know which users are likely to convert, so that follow-up spend can be directed to the most valuable prospects. It also needs to know which exposure factors are associated with conversion.

Objective. Build an interpretable predictive model of conversion and use regularization to improve its interpretability and, where possible, its performance. Specifically:

- Fit logistic regression models with L1 (Lasso) and L2 (Ridge) penalties.
- Tune the regularization strength with stratified cross-validation.
- Evaluate on a held-out test set with metrics suited to a heavily imbalanced target.
- Interpret the coefficients, showing how regularization changes feature selection and interpretability.

All analysis was done in Python in the accompanying Jupyter notebook. Every number in this report comes from that notebook's executed output.

2. Data Collection

Source. The “Marketing A/B Testing” dataset published on Kaggle by the user faviovaz (file marketing_AB.csv). It is the only data used in this project; no data were simulated or added. It contains 588,101 users, one row each, and 7 columns.

Column	Type	Description
user id	Integer	Unique user identifier
test group	Categorical	“ad” (saw product ads) or “psa” (saw a public-service announcement; control)
converted	Boolean	Target variable: whether the user bought the product
total ads	Integer	Number of ads the user saw during the campaign (exposure history)
most ads day	Categorical	Day of the week on which the user saw the most ads
most ads hour	Integer 0–23	Hour of the day at which the user saw the most ads
Unnamed: 0	Integer	Saved row index from the original export

Table 1. Dataset fields.

Scope note. The assignment suggests demographics and purchase history. This dataset instead describes campaign exposure: how many ads each user saw, and when. These are behavioural features, and the binary conversion target fits the marketing-prediction task. The absence of demographic variables is acknowledged as a limitation in Section 9.

Initial profile.

- **Imbalance:** 2.52% of users converted (14,843 of 588,101), roughly 1 converter per 40 users.
- **Group sizes:** 96% of users are in the ad group and 4% in the PSA group.
- **Conversion by group:** ad 2.55% vs PSA 1.79%, a 43.1% relative lift.

- **Ad exposure:** total ads is highly right-skewed (median 13, maximum 2,065). Conversion rises from 0.19% in the lowest exposure decile to 14.3% in the highest (Figure 1).
- **Timing:** Monday (3.28%) and Tuesday (2.98%) convert best and Saturday (2.11%) worst. Hours 1–4 a.m. convert worst (0.73% at 2 a.m.); afternoon and evening hours are near 3% (Figure 2).

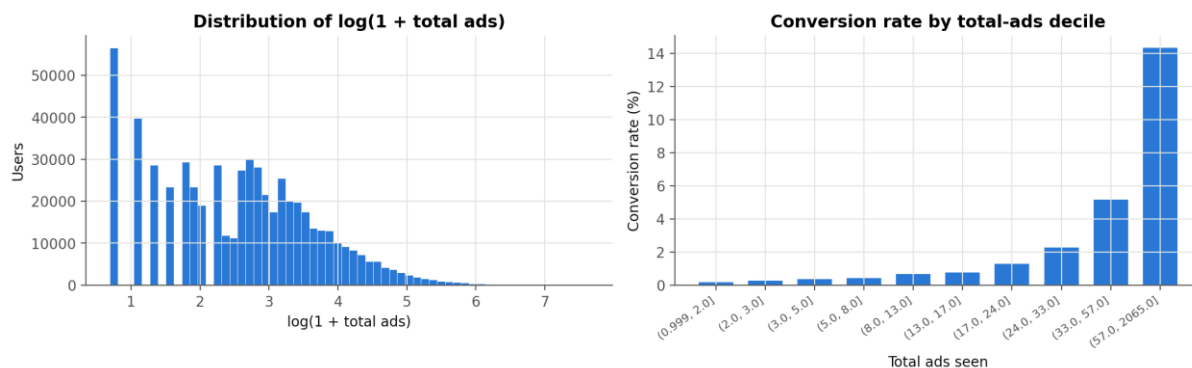


Figure 1. Distribution of $\log(1 + \text{total ads})$ and conversion rate by exposure decile.

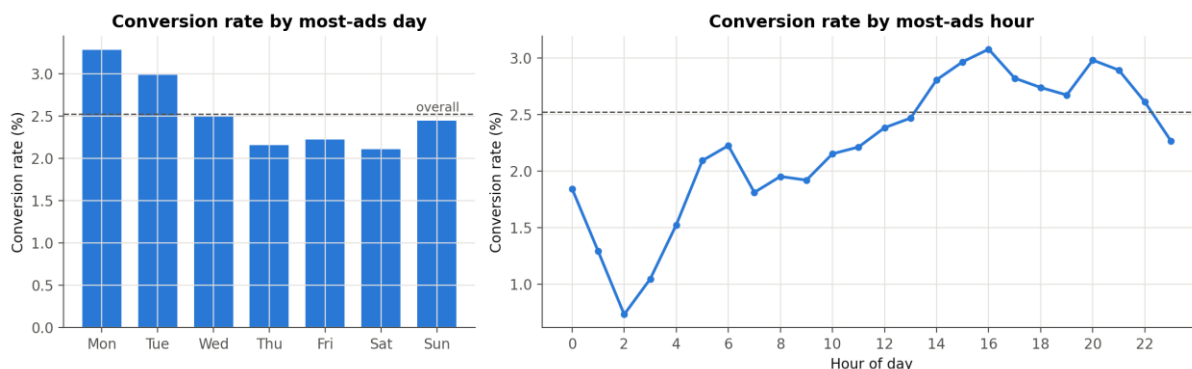


Figure 2. Conversion rate by most-ads day and hour (dashed line: overall rate).

3. Data Preprocessing

All fitted preprocessing steps live inside a scikit-learn Pipeline. Imputers, encoders and scalers are therefore re-fitted within each cross-validation fold, which prevents information from the validation folds or the test set leaking into training.

3.1 Data quality and missing values

The data-quality audit found:

- **no missing values** in any column;
- **no duplicate rows** and **no duplicate user IDs**;
- that “Unnamed: 0” is identical to the row index.

Imputers are still included so the model can score future data that does have gaps: median for numeric features, most-frequent for categorical ones. The identifiers (“user id” and “Unnamed: 0”) were dropped because they carry no predictive information and would only invite spurious fitting.

3.2 Feature engineering and categorical encoding

- **log_total_ads = log(1 + total ads):** compresses the extreme skew and matches the diminishing-returns shape seen in the EDA.
- **total_ads (raw count):** kept alongside its log on purpose. The two are strongly correlated, which shows how Lasso and Ridge handle redundant predictors differently.
- **is_ad:** 1 if the user was in the ad group, 0 for PSA.
- **Day and hour:** one-hot encoded. Hour is treated as categorical because its effect is not monotonic (Figure 2).
- **Encoding for regularized models:** every level is kept (full one-hot). With a penalty the model is identifiable without a dropped level, and keeping all levels avoids results that depend on an arbitrary reference category; hour 00, for example, holds under 1% of users.
- **Encoding for the unregularized baseline:** the first level is dropped (reference coding), because without a penalty a full set of dummies is perfectly collinear with the intercept.

3.3 Scaling

All numeric features were standardized to mean 0 and standard deviation 1 with StandardScaler. For the regularized models the one-hot dummies were standardized too, which is the default in R's glmnet. This step matters for regularization:

- The L1 and L2 penalties act on raw coefficient size, so features must share a scale for the penalty to treat them equally.
- Standardized coefficients can be compared directly as measures of importance.
- Centring the dummies separates them from the intercept, which let the L1 solver converge reliably.

The final design matrix has 34 features for the regularized models: 3 numeric, 7 day and 24 hour indicators. The baseline has 32 features.

3.4 Train/test split

The data were split 80/20 with stratification on the target: 470,480 training users and 117,621 test users, both with a 2.52% conversion rate. The test set was used only once, for the final evaluation.

4. Model Selection & Regularization

Model choice. The target is binary, so logistic regression is the natural linear model:

- it produces conversion probabilities, which can be used to rank users and allocate budget;
- its coefficients are directly interpretable as changes in the log-odds of conversion;
- it trains efficiently on more than half a million rows.

Regularization. In scikit-learn's form, the model minimizes a penalty plus C times the logistic loss, where $C = 1/\lambda$. A smaller C means a stronger penalty.

- **Ridge (L2):** the penalty is $\frac{1}{2}\|w\|_2^2$. All coefficients shrink smoothly toward zero and weight is shared among correlated features, but no coefficient becomes exactly zero, so Ridge does not select features.
- **Lasso (L1):** the penalty is $\|w\|_1$. Its corners drive weak coefficients exactly to zero, so feature selection is built in and the resulting models are sparser and easier to explain.

Candidate models.

Model	Role
Majority baseline	No-skill reference: always predicts the training conversion rate
Unregularized LR	Logistic regression with no penalty; shows what regularization adds
Ridge (L2)	L2 penalty with C tuned by cross-validation (lbfgs solver)
Lasso (L1)	L1 penalty with the C that maximizes cross-validated AUC (liblinear solver)
Lasso (L1, 1-SE)	L1 penalty with the strongest C whose CV AUC is within one standard error of the best (the one-standard-error rule; Hastie et al., 2009). Favours a sparser model.

Table 2. Models compared.

Solver note: liblinear also penalizes the intercept. Setting `intercept_scaling = 100` makes that penalty negligible, so the intercept is effectively unpenalized, as in the other models.

5. Training & Hyperparameter Tuning

Cross-validation setup.

- **Folds:** stratified 5-fold cross-validation on the training set only (shuffled, `random_state = 42`). Stratification keeps the 2.5% positive rate in every fold.
- **Grid:** 11 log-spaced values, $C \in \{10^{-4}, 10^{-3.5}, \dots, 10^1\}$, searched for each penalty with `GridSearchCV`. Exploratory runs showed CV performance is flat well before $C = 10$.
- **Selection metric:** ROC-AUC. It does not depend on a threshold, it is robust to class imbalance, and it measures how well the model ranks likely converters, which is what targeting needs. Log-loss was recorded as a secondary metric.
- **Threshold:** with 2.5% positives the default 0.5 cut-off almost never predicts a conversion. The operating threshold was therefore chosen to maximize F1 on out-of-fold training predictions, leaving the test set untouched.

C	Lasso AUC	SD	Ridge AUC	SD	Lasso log-loss	Ridge log-loss
0.0001	0.8534	0.0036	0.8561	0.0032	0.1105	0.1013
0.0003162	0.8534	0.0036	0.8562	0.0032	0.0978	0.0974
0.001	0.8537	0.0036	0.8566	0.0032	0.0958	0.0949
0.003162	0.8559	0.0034	0.8569	0.0032	0.0943	0.0941
0.01	0.8570	0.0033	0.8570	0.0032	0.0939	0.0939
0.03162	0.8570	0.0033	0.8570	0.0032	0.0939	0.0939
0.1	0.8570	0.0032	0.8570	0.0032	0.0939	0.0939
0.3162	0.8570	0.0032	0.8570	0.0032	0.0939	0.0939
1	0.8570	0.0032	0.8570	0.0032	0.0939	0.0939
3.162	0.8570	0.0032	0.8570	0.0032	0.0939	0.0939
10	0.8570	0.0032	0.8570	0.0032	0.0939	0.0939

Table 3. Stratified 5-fold CV results (mean across folds) for each C.

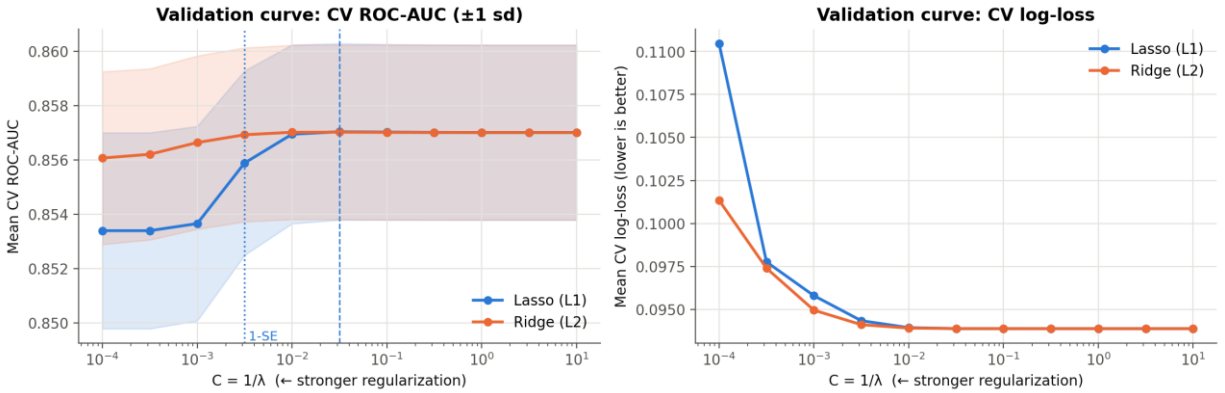


Figure 3. Validation curves. Dashed line: best Lasso C. Dotted line: Lasso C chosen by the 1-SE rule.

Tuning results.

- **Best C:** 0.0316 for Lasso (CV AUC 0.8570) and 0.0316 for Ridge (CV AUC 0.8570).
- **1-SE rule:** selects $C = 0.0032$ for Lasso, a penalty ten times stronger.
- **Plateau:** both curves flatten at about 0.857 from $C \approx 0.01$ onward.
- **Strong penalty:** at $C = 10^{-4}$ Lasso keeps only the ad-volume terms and still reaches a CV AUC of 0.853, a first sign that ad exposure carries most of the signal.
- **Little overfitting:** training and CV AUC never differ by more than about 0.0005. With about 470,000 training rows and 34 features, there is little variance for regularization to remove.

After tuning, each model was refit on the full training set.

6. Evaluation

Metrics. Accuracy is reported but is not informative here: a model that never predicts a conversion is 97.5% accurate. The analysis relies on:

- **ROC-AUC:** ranking quality across all thresholds.
- **PR-AUC (average precision):** more informative than ROC-AUC under heavy imbalance (Saito & Rehmsmeier, 2015); its no-skill value equals the base rate, 0.025.
- **Precision, recall and F1:** measured at the F1-optimal threshold.
- **Log-loss and Brier score:** quality of the predicted probabilities.
- **Top-decile capture:** the share of all converters reached by targeting the 10% of users with the highest scores.

Model	Thr.	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	Log-loss	Top-10%
Majority baseline	0.500	0.975	0.000	0.000	0.000	0.500	0.025	0.1178	9.7%
Unregularized LR	0.110	0.938	0.173	0.386	0.239	0.860	0.142	0.0935	57.8%
Ridge (L2)	0.107	0.936	0.171	0.401	0.240	0.860	0.142	0.0935	58.0%
Lasso (L1)	0.106	0.935	0.171	0.404	0.240	0.860	0.142	0.0935	58.0%
Lasso (L1, 1-SE)	0.096	0.931	0.164	0.422	0.236	0.860	0.138	0.0938	57.7%

Table 4. Held-out test-set performance (117,621 users) at each model's F1-optimal threshold. Top-10% = share of converters among the highest-scored decile.

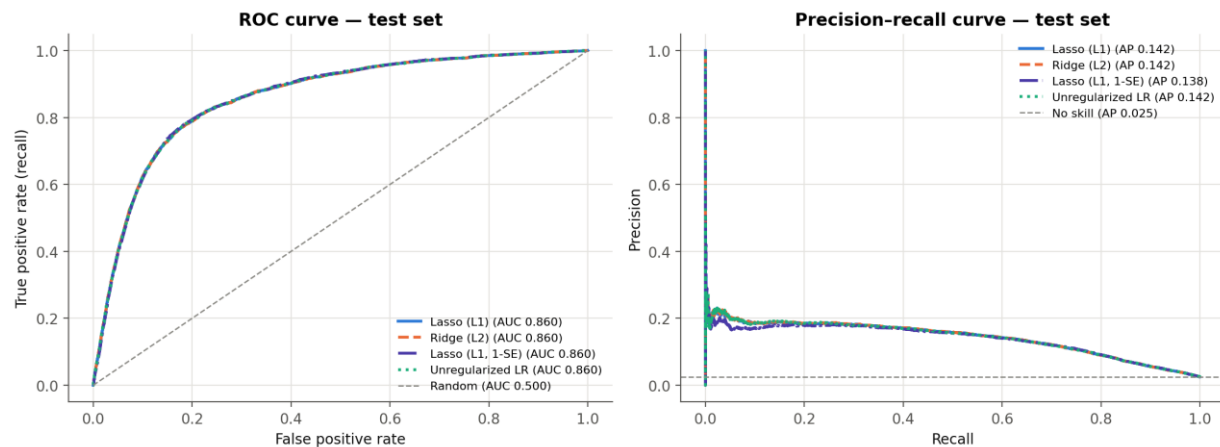


Figure 4. ROC and precision–recall curves on the test set.

Interpretation of performance.

- **Strong ranking:** every logistic model reaches test ROC-AUC ≈ 0.860 and PR-AUC ≈ 0.142 , about 5.6 times the no-skill level. The cross-validation estimate (0.857) closely matches the test result, confirming that the model generalizes.
- **Usable classification:** at its F1-optimal threshold (0.106), the tuned Lasso identifies 40.4% of converters with 17.1% precision (F1 = 0.240). The precision is about 6.8 times the rate expected from random selection. At the default 0.5 threshold every model has zero recall, which is why the threshold had to be tuned.
- **Regularization does not change accuracy here:** Lasso, Ridge and the unregularized model differ only in the third or fourth decimal place. The sparse Lasso 1-SE model uses 22 of 34 features and gives up almost nothing (ROC-AUC 0.860, PR-AUC 0.138).
- **Business value:** targeting the top 10% of users by score reaches 58% of all converters, a lift of 5.8 over random targeting (Figure 6).

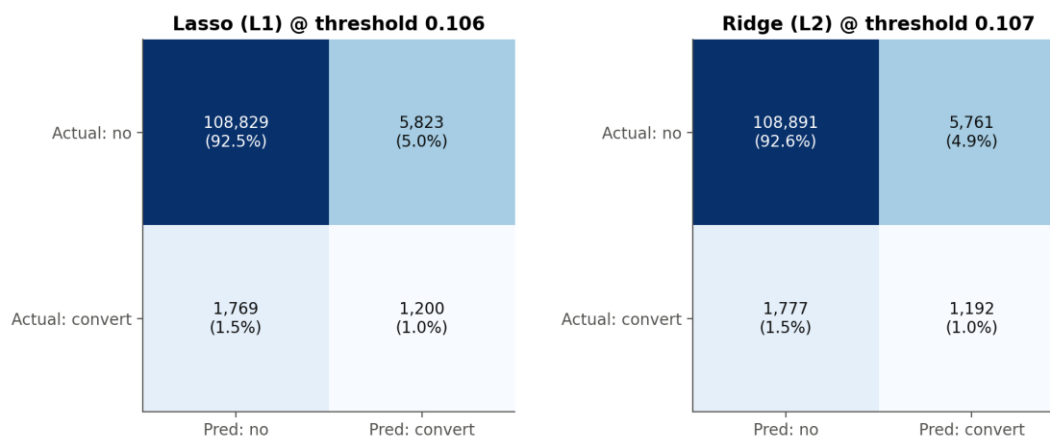


Figure 5. Test-set confusion matrices at the F1-optimal thresholds.

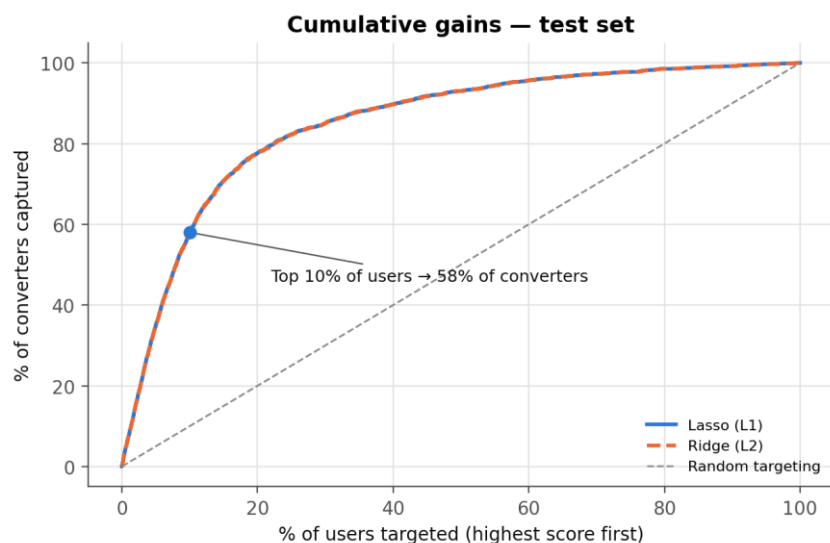


Figure 6. Cumulative gains chart on the test set.

7. Interpretation

All inputs were standardized, so each coefficient is the change in the log-odds of conversion for a one-standard-deviation increase in that feature. Because day and hour are fully one-hot encoded, a dummy's coefficient measures how far that level deviates from a typical day or hour. A Lasso coefficient of exactly zero means the level cannot be distinguished from typical.

Feature	Ridge	Lasso (best C)	Lasso (1-SE)	Kept by 1-SE?
log_total_ads	1.960	1.958	1.782	Yes
total_ads	-0.271	-0.271	-0.205	Yes
most_ads_hour_02	-0.126	-0.121	-0.058	Yes
most_ads_day_Monday	0.101	0.116	0.095	Yes
most_ads_hour_16	0.087	0.073	0.058	Yes
is_ad	0.082	0.078	0.045	Yes
most_ads_day_Tuesday	0.079	0.092	0.068	Yes
most_ads_hour_01	-0.075	-0.074	-0.031	Yes
most_ads_day_Saturday	-0.075	-0.057	-0.034	Yes
most_ads_day_Friday	-0.069	-0.050	-0.028	Yes
most_ads_hour_07	-0.061	-0.062	-0.029	Yes
most_ads_hour_09	-0.055	-0.062	-0.024	Yes
most_ads_hour_03	-0.052	-0.050	-0.008	Yes
most_ads_hour_15	0.047	0.032	0.017	Yes

Table 5. Standardized coefficients for the 14 largest Ridge effects. All 34 are in results/coefficients.csv.

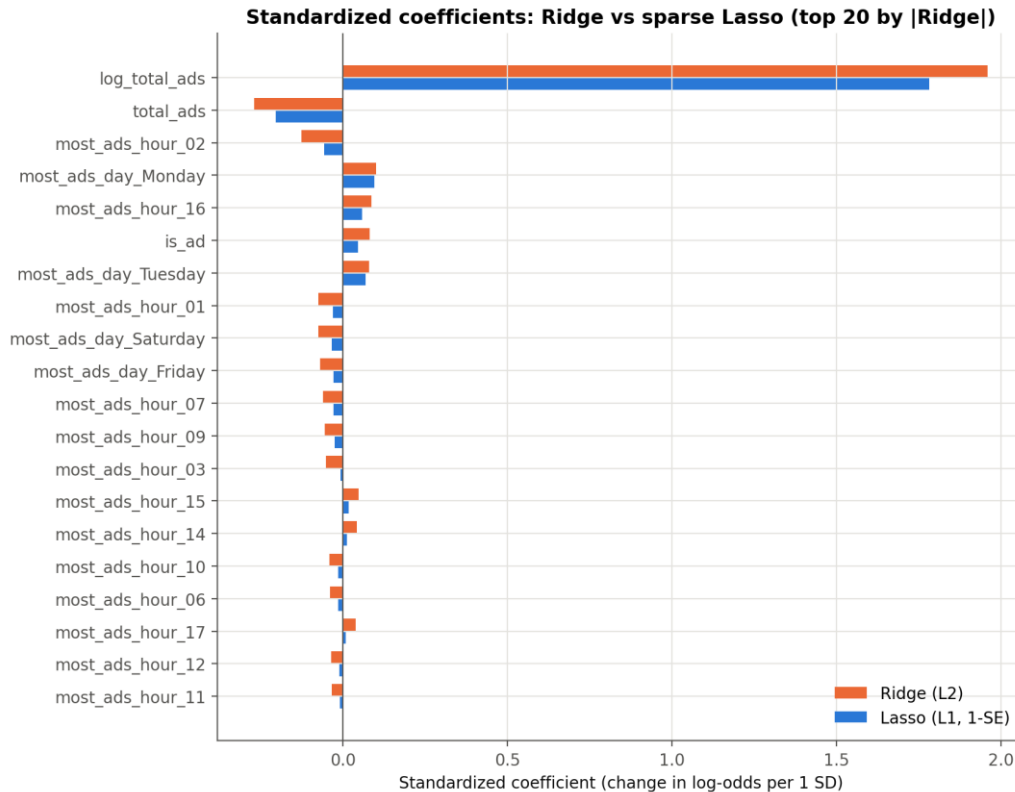


Figure 7. Ridge vs sparse Lasso (1-SE) coefficients.

7.1 Feature importance: what drives conversion

- **Ad exposure dominates.** log_total_ads has the largest coefficient by far (+1.96): the odds of converting multiply by about 7.1 for each one-standard-deviation increase in log exposure. The raw total_ads term is negative (-0.27), so combined with the log term the gains flatten at very high exposure.
- **Ad arm.** is_ad has a coefficient of +0.078 per SD. On the 0/1 scale that is about +0.40 log-odds, or roughly 1.5 times the odds of converting compared with PSA users, holding exposure and timing fixed. The data come from an A/B test, so if assignment was randomized this coefficient has the strongest causal reading. The other coefficients describe associations only.
- **Day of week.** Monday (+0.12) and Tuesday (+0.09) are above typical; Friday and Saturday are below.
- **Hour of day.** Early-morning hours are below typical, with 2 a.m. the most negative (-0.12), followed by 1 a.m. and 7 a.m. Mid-afternoon (2–5 p.m.) is above typical, with 4 p.m. the strongest (+0.07). This is directly usable for ad scheduling.

7.2 Impact of regularization on feature selection and interpretability

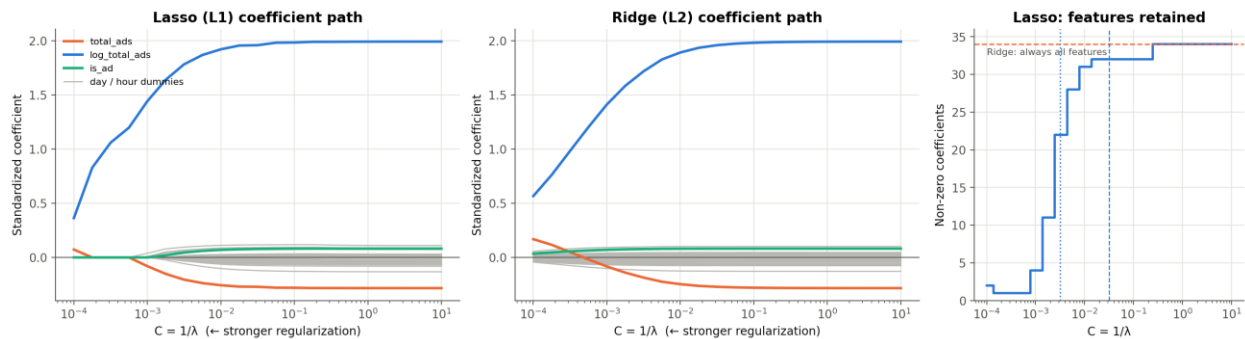


Figure 8. Coefficient paths for Lasso and Ridge, and the number of features Lasso retains, as C varies.

- **Ridge keeps every feature.** All 34 coefficients stay non-zero at every C . Ridge is stable and predicts well, but every one of the 31 day and hour indicators still has to be explained.
- **Lasso at the best C is almost unpenalized.** It sets only 2 coefficients to zero (Sunday and hour 19), because with this much data a weak penalty is optimal.
- **Lasso with the 1-SE rule selects features.** It keeps 22 of 34 features, and its total coefficient size (L1 norm) is 69% of the least-regularized fit, with no material loss in test performance. It removes Wednesday, Thursday and Sunday, and hours 00, 04, 13 and 18–23.
- **What the removals mean.** Evening hours look strong in the raw conversion rates (Figure 2), yet Lasso removes them. Once ad volume is accounted for, those hours add little extra information, which the unregularized coefficients alone would not make obvious.
- **Correlated features.** At strong penalties (C between about 2×10^{-4} and 8×10^{-4}), Lasso keeps only `log_total_ads` and sets `total_ads` exactly to zero, while Ridge keeps both and shares the weight between them. At the tuned C both models keep both terms, because the raw count adds the flattening-at-high-exposure effect.
- **Order of importance.** The Lasso path ranks features by how early they enter as the penalty weakens: ad volume first, then Monday and Tuesday, then Friday, Saturday, the ad-arm flag and the early-morning hours.

Summary of the sparse model. Conversion is driven by ad volume (with diminishing returns), being in the ad arm, Monday and Tuesday exposure, mid-afternoon peaks and early-morning lows. Its ranking performance is the same as the full model's.

8. Conclusion

Findings.

- **Performance:** regularized logistic regression predicts conversion well, with test ROC-AUC 0.860 and PR-AUC 0.142. Targeting the top-scored decile reaches 58% of converters.
- **Tuning:** cross-validation found a broad plateau of good C values. Because the dataset is large relative to the number of features, regularization does not improve accuracy over an unregularized model.

- **Value of regularization:** its benefit here is interpretability. The Lasso 1-SE model removes 12 of 34 features at no material cost, the L1 path ranks features by robustness, and the comparison with Ridge shows how each penalty handles correlated predictors.
- **Business takeaways:** ad volume is the main lever, with diminishing returns; real ads are associated with about 1.5 times the conversion odds of PSAs; and scheduling should favour Monday, Tuesday and mid-afternoon over the early-morning hours.

9. Limitations and Next Steps

Limitations.

- The dataset has no demographic or purchase-history variables.
- `total_ads` is a campaign-level count. The data do not show whether it includes ads seen after conversion, so part of the exposure effect may reflect reverse causation: engaged users may both see more ads and buy.
- Only 4% of users are in the PSA group, so effects specific to that group are estimated less precisely.

Next steps.

1. Add demographic and purchase-history features (recency, frequency, monetary value) if they become available.
2. Try Elastic Net, which keeps groups of correlated features together, and group Lasso, which selects whole variables such as all 24 hours at once.
3. Benchmark gradient boosting to measure non-linear gains, keeping the Lasso as the interpretable reference.
4. Set the operating threshold from campaign economics (cost per contact vs. margin per conversion).
5. Use uplift modelling on the two A/B arms to target users whose conversion is caused by the ads.

References

- faviovaz. (n.d.). Marketing A/B testing [Data set]. Kaggle. Retrieved September 30, 2026, from <https://www.kaggle.com/datasets/faviovaz/marketing-ab-testing>
- Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R., & Lin, C.-J. (2008). LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9, 1871–1874.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). Jupyter Notebooks – a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing* (pp. 87–90). IOS Press.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.

Software: Python 3.11, pandas 2.3, NumPy 2.3, scikit-learn 1.7, Matplotlib 3.9, Jupyter.